

TWIN Platform Overview

TWIN is a composable digital infrastructure platform enabling trusted, policy-controlled data exchange between organisations across any sector. The hard problems - decentralised identity, tamper-evident audit trails, data governance, and cross-organisation sharing - are solved once at the platform level and reused without rebuilding across every application constructed on top of it.

TWIN-Core: The Platform Foundation

TWIN-Core is the engine underneath every TWIN application. At over **583,000 lines of TypeScript** - 91% of the total ecosystem codebase - it provides 35 ready-to-use capability modules spanning identity, audit, data governance, cross-organisation integration, and enterprise infrastructure. No application built on TWIN-Core re-implements any of this; it inherits it.

The defining architectural decision is the **connector pattern**: every capability is defined as a clean interface, with one or more swappable backend implementations behind it. Application logic is written against the interface. The backend - whether a cloud database, a distributed ledger, or an in-memory store - is a deployment configuration. This makes TWIN applications genuinely portable across infrastructure without code changes, and it is what makes the platform vertical-agnostic: the same 35 modules that serve logistics serve healthcare, finance, or carbon accounting equally well.

583k

Lines of
TypeScript

35

Capability
modules

9+

Database
backends

14+

International
standards

IDENTITY & TRUST

Full self-sovereign identity stack 9 packages

TWIN-Core implements the complete W3C Decentralised Identity (DID/SSI) specification. Every participant in a TWIN network has a **Decentralised Identifier** - a globally unique, cryptographically verifiable identity document that no central

authority controls. Identities can be anchored to the IOTA distributed ledger for tamper-evident on-chain resolution, or to entity storage for lightweight deployments.

On top of identity, TWIN-Core issues and verifies **Verifiable Credentials** - digitally signed attestations that encode trust relationships without requiring the relying party to contact the issuer. Supporting capabilities include NFT minting and burning for asset ownership proof, cryptographic notarisation of documents, and immutable proofs that can be attached to any business record to make its provenance verifiable at any future point.

twin-identity

twin-dlt

twin-wallet

twin-vault

twin-attestation

twin-nft

twin-notarization

twin-immutable-proof

twin-trust

AUDIT & DATA

Tamper-evident records and data processing

7 packages

TWIN-Core provides two complementary data structures for building auditable business records. The **Auditable Item Stream** is an immutable, ordered event log - every entry is cryptographically chained to the one before it, making any retroactive modification detectable. The **Auditable Item Graph** extends this to relationship tracking: it stores connected entities and their state changes with the same tamper-evident guarantees, enabling a complete verifiable history of who did what, to what, and when.

Above these primitives sit document management (versioned storage with audit trails attached to graph records), structured data validation with JSON-LD processing and JSON Path querying, a data extraction and format conversion pipeline, and hierarchical data management with both DLT and entity storage backends.

twin-auditable-item-stream

twin-auditable-item-graph

twin-document-management

twin-data

twin-data-processing

twin-standards

twin-hierarchies

DATA SHARING & INTEGRATION

Cross-organisation exchange with policy control

3 packages

TWIN-Core implements the **Eclipse Dataspace Protocol** (IDSA/DSP) - the emerging international standard for sovereign, policy-controlled data exchange between organisations. The implementation covers the full protocol: a control plane for negotiating data-sharing agreements, a data plane for the actual transfer, and a

WebSocket client for real-time push. Organisations share data on their own terms; the platform enforces those terms automatically.

Access policies are expressed in **ODRL** (W3C Open Digital Rights Language) and enforced at five points in the data lifecycle: Administration, Decision, Enforcement, Negotiation, and Exchange. The federated catalogue module allows organisations to publish dataset metadata and discover what others are offering - the discovery layer for the dataspace network.

twin-dataspace

twin-rights-management

twin-federated-catalogue

INFRASTRUCTURE

Production-grade runtime and observability

15 packages

The largest capability domain by package count, TWIN-Core's infrastructure layer provides everything needed to run a TWIN application in production. The **engine** manages application lifecycle - startup, shutdown, health checks, service registration, and configuration. The **API** layer provides a full Fastify-based REST server with contract-first definitions, generated clients, and multi-tenant context handling out of the box.

Storage is covered at two levels: entity storage across **9 backends** (CosmosDB, DynamoDB, MongoDB, MySQL, PostgreSQL, ScyllaDB, GCP Firestore, file, and memory) and binary/blob storage across AWS S3, Azure Blob, GCP Storage, and IPFS. Secret and key management runs through HashiCorp Vault or entity storage. Observability is built on **OpenTelemetry** throughout - structured logging, metrics, and distributed tracing are first-class concerns, not afterthoughts. Background task scheduling, event routing with WebSocket support, synchronised storage across distributed deployments, and automated workflow execution round out the layer.

twin-engine

twin-api

twin-framework

twin-node

twin-entity-storage

twin-blob-storage

twin-vault

twin-logging

twin-telemetry

twin-tracing

twin-event-bus

twin-background-task

twin-automation

twin-synchronised-storage

twin-tools

STANDARDS COMPLIANCE

14+ international vocabularies built in

TWIN-Core ships TypeScript type definitions for 14 international standards and vocabularies, covering W3C specifications (DID, ODRL, DCAT, Activity Streams), global trade data models (GS1, UN/CEFACT, DCSA, VDA), and semantic web

foundations (Dublin Core, FOAF, Schema.org). These are not adapters added as an afterthought - they are first-class types in the platform's data model.

The practical consequence is that data produced by one TWIN application is structurally interoperable with any other system that implements the same standards - without custom translation layers. This is the foundation of the network effect: every new participant speaks the same language by default.

TWIN Platform Stack

PRESENTATION LAYER

Supply Chain Frontend

Next.js application · TWIN ID identity portal · apps/frontend

DOMAIN APPLICATION LAYER

Supply Chain Backend

Service packages · Mobius adaptor · Dataspace connector · packages/ & apps/supply-chain-node

PLATFORM FOUNDATION LAYER

TWIN-Core

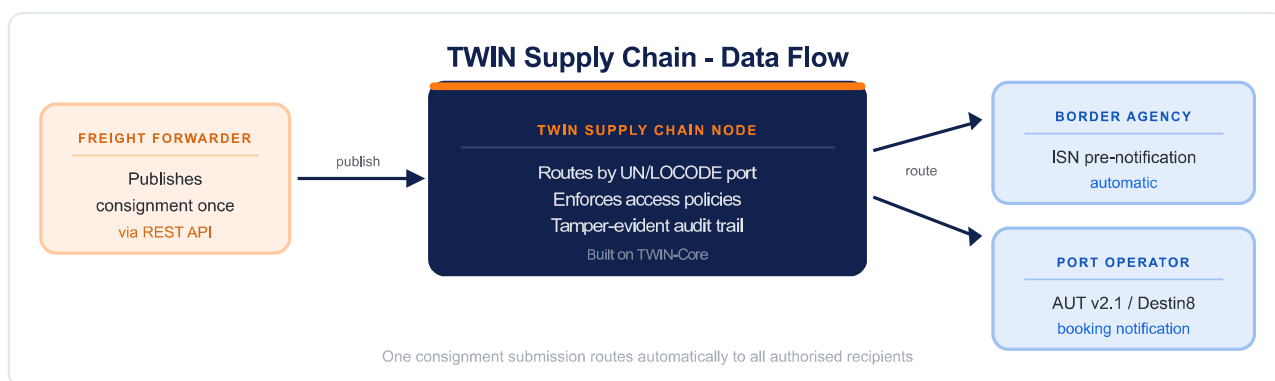
Identity · Audit · Data governance · Integration · Infrastructure

35 packages · 583,071 lines of TypeScript · 9+ database backends · 14+ standards

Supply Chain: A Reference Application

The TWIN supply chain application demonstrates the platform model in production. At roughly **59,000 lines** across its backend services and frontend, it sits on top of TWIN-Core's 583,000 - contributing all logistics-specific intelligence while re-implementing none of the platform layer. Every identity, audit trail, policy, and data-sharing capability it uses is inherited directly from TWIN-Core.

The backend solves a concrete problem in international freight: a Freight Forwarder publishes a consignment once via the supply chain REST API, and the platform routes the correct pre-notification automatically to border agencies (ISN format) and port operators (AUT v2.1 / Destin8), based on the consignment's UN/LOCODE ports of entry and exit. A Mobius adaptor bridges to the incumbent Mobius freight-forwarder platform. The deployment is multi-tenant - a single node serves multiple freight forwarder organisations with fully isolated identity, data, and access policies.



The frontend is a Next.js application providing a consignment visibility portal for logistics operators. It is intentionally a thin presentation layer - consuming the supply chain REST API with no platform logic of its own, which means the same TWIN-Core node can equally serve third-party systems or alternative front ends without modification.

Key Strengths

Vendor-neutral infrastructure

The connector architecture prevents lock-in to any cloud provider, database, or ledger technology. Infrastructure choices evolve without changing application logic.

Standards-based interoperability

W3C Decentralised Identifiers, Verifiable Credentials, Eclipse Dataspace Protocol, GS1, and UN/CEFACT ensure TWIN data is interoperable with any conformant system globally - without custom translation.

Composable across verticals

The same 35 building blocks that power supply chain today can power healthcare, finance, or carbon accounting tomorrow. Each new vertical requires only domain-specific logic on top - the 583,000 lines of TWIN-Core carry across unchanged.

Production proven

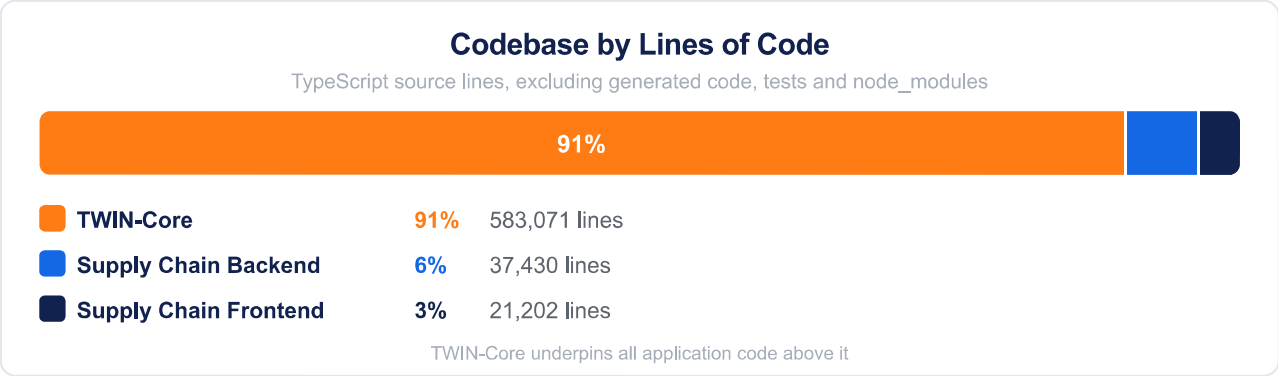
Already deployed in a UK government-backed supply chain pilot with live freight forwarder organisations.

Network effects

Each organisation that joins a TWIN dataspace increases the value of the network for all participants. The platform becomes more valuable with every new node connected.

By the Numbers

Measured in actual lines of TypeScript source code (excluding generated code, tests, and dependencies), TWIN-Core accounts for **91% of the total ecosystem codebase**. The entire supply chain application - backend services and frontend combined - represents the remaining 9%.



A new vertical application built on TWIN-Core inherits over half a million lines of battle-tested infrastructure. The incremental cost of that vertical is the domain-specific logic only - comparable to the supply chain's combined ~59,000 lines, regardless of the underlying platform capabilities it gains for free.